マルウェア動的解析における対話的イベント応答のための VLMベースエージェントの設計と評価

塙 剛生[1,a)]　藤井 翔太[1]

概要：マルウェアには，ダイアログのボタンクリックなどの対話的な操作の要求（以下，イベントと呼ぶ）に応答することによって動作が進行するものが存在する．このため，マルウェア動的解析を自動化するには，各イベントに対して適切な操作を自動で実行する必要がある．他方で，そうしたイベントは多種多様であり，ルールに基づく現行のマルウェア動的解析の自動化システムでは，網羅的に対応することが現実的ではない．そこで本研究では，マルウェア動的解析における対話的イベント応答を自動化するため，視覚言語モデル（Vision Language Model: VLM）を用いたイベント応答エージェントを提案する．提案手法は，解析環境のスクリーンショットと操作履歴を基に，VLM が次に実行すべき操作を生成し，解析環境上で自動実行する．また，画面遷移前に取得した情報を履歴として保持し，後続画面での操作生成に反映することで，複数画面にまたがるイベント応答を可能にする．評価では，実際にマルウェア動的解析中に確認されたイベントを基に 13 件の模擬検体を作成し，CAPE v2 上で提案手法を実装して性能を検証した．その結果，提案手法は 39 試行中 31 試行 (79.49%) で，解析に必要な一連の操作を完了することを確認した．また，失敗事例分析により，本エージェントの課題と今後の研究可能性を導出した．

Design and Evaluation of VLM-based Agent for Interactive Event Response in Malware Dynamic Analysis

Goki Hanawa[1,a)]　Shota Fujii[1]

Abstract: Some malware activates behavior by responding to interactive requests such as dialog button clicks. Therefore, automating dynamic malware analysis requires automatic execution of accurate actions for each event. However, traditional rule-based systems struggle with malware that demands actions beyond predefined rules. To address this limitation, we propose an event-response agent using a Vision Language Model (VLM) to automate interactive event responses in dynamic malware analysis. The proposed method uses screenshots and operation history from the analysis environment to generate and automatically execute the next required operation. For evaluation, we created 13 simulated samples based on events identified in malware dynamic analysis and implemented the proposed method on CAPE v2. As a result, the proposed method successfully completed the sequence of operations required for the analysis in 31 out of 39 trials (79.49%). Furthermore, we summarized the remaining challenges of the agent and future research directions through failure case analysis.

1. はじめに

マルウェア動的解析は，解析環境下でマルウェア検体を実際に実行させることでその挙動を明らかにする解析方法である．実際の組織では日々多数のマルウェア検体が着弾するため，限られた人員ですべての検体を手作業で解析するのは現実的ではない．このため，解析者の手動解析に加え，サンドボックスなどの自動解析を導入し，基本的な解析を自動実行することで全体の効率化が図られている [1].

一方で，マルウェアの中には，ダイアログのボタンクリックや ClickFix でのキー入力などの操作要求（以下，イベントと呼ぶ）に適切に応答した場合にのみ，動作が進行して悪性行動に至るものが存在する．このようなマルウェア動

[1] 株式会社日立製作所 Hitachi, Ltd.
[a)] goki.hanawa.rc@hitachi.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

的解析中に表示されるイベントは，表示形式や操作内容等が多様であるのに対して，従来の解析環境における自動操作は，特定の文字列を含むボタンをクリックするなど，単純なルールベースの処理に依存している．そのため，ルール外のイベントや複数画面にまたがる操作，画面上の自然言語指示に基づく操作には十分に対応できない．

このような背景を受け，本研究では，マルウェア動的解析における対話的イベント応答を自動化するため，視覚言語モデル（Vision Language Model：VLM）ベースエージェントを提案する．提案手法は，解析環境のスクリーンショットと，過去の画面情報，生成・実行した操作，および操作検証結果を含む実行履歴を基に，VLM を用いて次に実行すべき操作列を生成し，解析環境上で自動実行する．操作後には，操作前後の状態を基に操作による状態変化の有無を検証し，その結果を実行履歴に追加する．提案手法は，この画面観測，操作生成，操作実行，操作検証，および履歴更新からなる処理を反復することで，解析中の画面状態や過去の実行結果を踏まえて後続の操作を決定し，複数画面にまたがるイベントへの継続的な応答を可能にする．本研究では，実運用に即した環境で提案手法を評価するため，CAPE v2 Sandbox（以降，CAPE v2） [2] 上に提案手法を構築した．性能評価では，実際のマルウェア動的解析中に確認されたイベントを基に模擬検体を作成し，それらを用いて提案手法の性能を検証した．さらに，成功事例および失敗事例を分析することで，提案手法の有効性と，課題および今後の研究可能性を導出した．

本稿の主要な貢献は以下の三点である．

- マルウェア動的解析中に表示される対話的イベントへ自動応答するため，VLM ベースイベント応答エージェントを設計し，CAPE v2 上に実装する方法を示した．
- 実際のマルウェア動的解析で確認されたイベントを再現した 13 件の模擬検体による評価データセットを構築した．評価した結果，提案手法は 39 試行中 31 試行（79.49%）で，解析に必要な一連の操作を完了した．
- 失敗事例を分析し，クリック領域の座標推論の誤り等の課題を抽出した．また，プロンプト設計やエージェント構成に関する今後の研究可能性を導出した．

2. 研究背景

2.1 マルウェアの動的解析におけるイベント応答

マルウェアの動的解析は，検体を実際に動作させ，その挙動を観察することでマルウェアの性質や機能を明らかにする解析方法である．マルウェアの中には，ユーザによる対話的操作を契機として後続の処理を進行させるものが存在するため，既存の動的解析環境では，イベントに自動応答するための機能が導入されている．例えば，CAPE v2 では，事前に作成されたキーワードに基づくダイアログ内のボタンの自動クリック機能（human.py）[*1]が導入されている．DRAKVUF [3] では，キーワードベースでの自動クリック機能[*2]が実装されている [4]．また，Joe Sandbox [5] では，フィッシングサイトの解析時に画面情報からボタンやリンクの位置・形状を認識することによる自動クリックが実現されている [6]．ただし，これらの既存手法はクリック操作の自動化に留まっている．このため，より広範かつ複雑な操作を要するイベントへの応答が課題である．

この課題に対して，著者らは前段の研究として，マルウェア動的解析中に要求されるイベントの体系的な調査を実施した [7]．ANY.RUN [8] 上に公開された 154 件の解析レポートを対象にコーディングを実施し，解析中に表示されたイベント画面と，解析を進行させるために必要な操作を抽出した．その結果，イベント種別は 14 種類と多岐にわたり，必要な操作もクリック操作以外を含む複数の操作種別に及ぶことが確認された．本研究では，前段の研究で確認されたイベント種別および操作種別を基に，イベント応答のための VLM ベースエージェントを設計する．これにより，多様なイベントへの応答を実現し，動的解析自動化のカバレッジ拡大を目指す．

2.2 VLM ベースの操作エージェントに関する研究

VLM ベースの操作エージェントは，スクリーンショットとタスク指示を基に画面状態と操作対象を認識し，マウスやキーボード操作を選択することで，タスクを自動化できることが示されている．Niu ら [9] は，デスクトップ環境を対象とした ScreenAgent を提案し，スクリーンショットの観測，操作生成，操作実行，実行結果の検証を繰り返すことで，スライド編集などの複数ステップのタスクを実行できることを示した．Lin ら [10] は，Web およびモバイル環境を対象とした ShowUI を提案し，スクリーンショットから操作対象を特定して，クリックや文字入力を含む操作を実行できることを示した．一方，既存研究は，実行すべきタスクや目標状態があらかじめ定義された一般的なアプリケーション操作を主な対象としており，マルウェア動的解析中に発生する多様なイベントへの適用可能性は明らかでない．そこで本研究では，VLM ベースのエージェントを構築し，イベント応答への適用可能性を検証する．

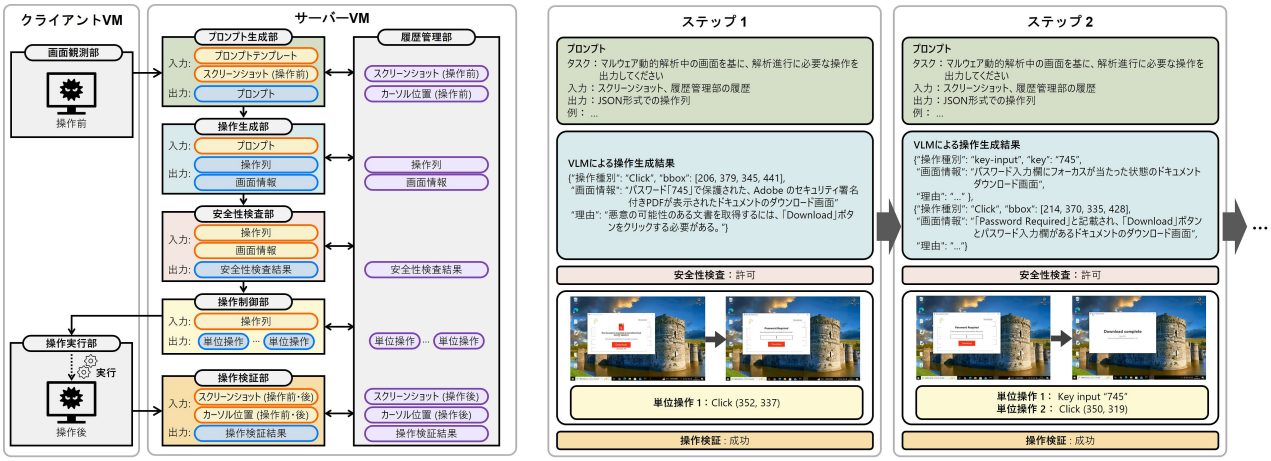
3. 提案手法

本章では，VLM ベースのイベント応答エージェントを提案する．本研究では，CAPE v2 におけるイベント応答機能（human.py）を提案手法に置換する前提で設計した．

[*1] https://github.com/cuckoosandbox/cuckoo/blob/master/cuckoo/data/analyzer/windows/modules/auxiliary/human.py

[*2] https://github.com/tklengyel/drakvuf/blob/main/src/plugins/hidsim/gui/vmi_win_gui_filter_definitions.h

© 2026 Information Processing Society of Japan



(a) 提案手法の処理フロー　　(b) 提案手法の動作例

図 1　本研究の全体像

3.1　全体像

提案手法の全体像を**図 1** に示す．提案手法は，CAPE v2 を基盤とする動的解析環境に組み込まれ，解析中に表示されるイベントを観測し，応答操作を自動実行する．CAPE v2 は，検体の投入，解析制御，挙動記録，レポート生成を担うサーバー側と，検体を実行するクライアント側から構成される．提案手法では，サーバー側が VLM による操作生成，状態管理，安全性検査，操作結果の検証を担い，クライアント側が画面状態の取得とイベント応答操作を担う．

提案手法は，画面観測，操作生成，安全性検査，操作実行，操作検証，履歴更新を一つの処理単位として反復する．まず，クライアント側で現在のスクリーンショットとカーソル位置を取得し，サーバー側へ送信する．サーバー側では，履歴管理部がスクリーンショット，VLM 出力，実行済み操作列，安全性検査および操作検証の結果を管理する．プロンプト生成部は現在の画面情報と履歴情報を統合し，VLM への入力を生成する．操作生成部は，このプロンプトとスクリーンショットから，操作列，および選択根拠を含む操作内容と，画面情報を出力する．安全性検査部は生成された操作列の出力形式，操作種別や変数の整合性などを実行前に検査する．検査で拒否された場合は操作を実行せず，判定結果を履歴に保存して次ステップへ移る．許可された場合は，操作制御部が操作列を実行可能な単位操作へ分解し，操作実行部へ順次送信する．操作実行部は，受信した単位操作に基づき操作を実行する．実行後は，再度スクリーンショットとカーソル位置を取得し，操作検証部が操作前後の状態を比較して操作の成否を判定する．操作内容等の情報は履歴に保存され，次ステップの操作生成時に参照される．この反復を繰り返すことで，提案手法は履歴情報を参照しながら次の操作を継続的に生成・実行する．詳細な処理アルゴリズムは付録 A.1 に示す．

3.2　画面観測部

画面観測部は，クライアント環境のスクリーンショットとカーソル位置を取得し，サーバー側へ送信する．提案手法では，各ステップの開始時と操作実行後に画面観測を行い，取得した観測結果を履歴管理部に保存する．

3.3　履歴管理部

履歴管理部は，後続の操作生成に用いる履歴情報とエージェントの実行状態を管理する．先行研究では，過去の操作履歴を利用して次の操作を計画する構成が用いられ，その有効性が示唆されている [11]．本研究においても，対話的イベントは単一の操作で完結するとは限らず，過去に実行した操作，操作前の画面から取得した情報，および操作後の状態変化が，次に実行すべき操作列の判断に影響し得る．そのため，本手法では，VLM が過去の履歴情報を参照できるように設計する．具体的には，履歴管理部にて，操作内容，画面情報および操作検証部の判定結果を履歴情報として保持する．これらの情報は次回のプロンプト生成部に渡され，VLM が過去の実行状況を踏まえて次の操作を生成するために利用される．また，履歴管理部は，スクリーンショット，総ステップ数や安全性検査および操作検証の連続失敗回数などの実行状態も保持する．これらの実行状態はエージェントの制御に用いられ，履歴情報とは区別して管理する．以上により，提案手法は，現在の画面だけでは取得できない過去の履歴情報を参照しながら，複数画面にまたがるイベントへの応答を継続できる．

3.4　プロンプト生成部

プロンプト生成部は，VLM に入力するプロンプトを生成する．本研究では，マルウェア動的解析中のイベントに依らず取得可能な情報であるスクリーンショットのみを観測空間として用いる．プロンプトには，マルウェア解析者としての役割，タスク説明，アクション空間および履歴管

© 2026 Information Processing Society of Japan

表 1 アクション空間

操作種別	変数
（左）クリック	bbox（x_1, y_1, x_2, y_2：整数）
右クリック	bbox（x_1, y_1, x_2, y_2：整数）
ダブルクリック	bbox（x_1, y_1, x_2, y_2：整数）
スクロール	方向（上／下）
キー入力	テキスト（文字列）
キー押下	キー（文字列）
ホットキー入力	ホットキー（文字列）
待機	なし

理部が保持する履歴情報を含める．これにより，VLM は現在画面の情報だけでなく，過去の情報を参照して次の操作を生成できる．詳細なプロンプトは付録 A.2 に示す．

本研究におけるアクション空間を**表 1** に示す．アクション空間は，前段の研究 [7] で確認された操作種別および先行研究 [9] を参考に，8 種類の操作を定義した．待機操作は，画面上に操作すべきイベントが存在しない場合に，追加操作を行わず状態変化を待機するために定義する．

3.5 操作生成部

操作生成部は, 生成されたプロンプトとスクリーンショットを VLM に入力し，次に実行すべき操作列を生成する．出力には，現在の画面情報，操作種別，座標などの変数，および推論根拠を含める．画面情報を履歴管理部に保存することで，VLM による画面解釈と一時的な表示情報を保持し，後続の操作生成に活用する．

3.6 安全性検査部

安全性検査部は，操作生成部が生成した操作列を実行前に検査し，クライアント環境での実行可否を判定する．先行研究 [12] では，生成操作を安全ルールに基づいて事前検査する重要性が示されており，本研究でも VLM の出力に同様の検査を適用する．

安全性検査は，静的フィルタと LLM フィルタの 2 段階で行う．静的フィルタでは，不正な JSON，未対応の操作種別，画面外座標などをルールベースで検出し，実行不能な操作列を除外する．LLM フィルタでは，操作内容やその推論根拠を踏まえて拒否ポリシーと照合し，意味的な安全性を判定する．拒否対象には，外部ネットワークや他端末への接続，端末外へのデータ送信など，クライアント VM 外へ影響し得る操作を含める．ただし，許容可能な操作は検体，解析目的，解析方針によって異なるため，拒否ポリシーは解析環境に応じて変更可能とする．いずれかの検査に失敗した操作列は実行せず，判定結果と理由を履歴に保存し，次回の操作生成に利用する．

3.7 操作制御部

操作制御部は，安全性検査で許可された操作列を単位操作に分解し，クライアント側の操作実行部へ順次送信する．

表 2 実装環境

役割	構成	CPU	メモリ
ホスト端末	OS: Ubuntu 24.04.4 LTS VMM: QEMU 8.2.2 / KVM (Linux kernel: 6.17.0-23)	AMD Ryzen 7 255	64 GB
サーバー VM	OS: Ubuntu 24.04.4 LTS	同上（vCPU 4 コア）	16 GB
クライアント VM	OS: Windows 10 Pro	同上（vCPU 2 コア）	4 GB

操作生成部は，入力欄のクリック後に文字列入力するように，複数の操作を一度に生成する場合があるため，操作制御部が実行順序を保持しながら各操作を一つずつ送信する．

3.8 操作実行部

操作実行部は，操作制御部から受信した単位操作をアクション空間で定義された操作へ変換し，クライアント環境上で実行する．クリック系操作では，対象要素を囲む矩形領域（bounding box; bbox）の中心座標を実行位置とし，キーボード操作では指定されたキーや文字列を入力するよう操作を変換する．操作制御部が実行順序を管理し，操作実行部が具体的なマウス・キーボード操作の実行を担うことで，生成した操作列を CAPE v2 上で自動実行する．

3.9 操作検証部

操作検証部は，操作実行後に，対象操作が実際に反映されたかを判定する．先行研究では，操作前後のスクリーンショット差分による検証が用いられている [13]．本研究ではこれに加え，入力欄やコマンドラインウィンドウのアクティブ化など，画面差分が生じにくい操作に対応するため，カーソル位置の変化も検証に用いる．検証結果は履歴管理部に保存され，次ステップのプロンプト生成に反映されることで操作の成否を踏まえた後続操作の生成を可能にする．

4. CAPE v2 への実装

4.1 実装環境

表 2 に本研究で用いた実装環境を示す．ホスト端末の VMM 上にサーバー VM とクライアント VM を構築する．サーバー VM には CAPE v2 を導入し，既存機能で検体投入，解析ジョブ管理，結果保存等を行う．また，提案手法の主要コンポーネントをサーバー VM 上に配置し，クライアント VM から取得した画面情報を基に VLM 推論を行う．クライアント VM には CAPE v2 の解析用 Windows 環境を構築し，マルウェア検体の実行に加えて，スクリーンショット取得と自動操作を行う．

4.2 CAPE v2 における機能追加

本研究では，CAPE v2 が備える既存フローを維持しつつ，human.py から提案手法に置換・拡張する．CAPE v2 では，カスタマイズ用の auxiliary module を活用することで機能拡張が可能である．本研究では，CAPE v2 の解析フローを維持したまま提案手法に置換するため，

© 2026 Information Processing Society of Japan

表 3　human.py と提案手法の評価検体別性能比較

評価検体	Completion Rate					Progress Rate					Grounding Rate	
	human.py		提案手法		差分	human.py		提案手法		差分	提案手法	
analysis.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/5)	**100%**	**(15/15)**	**+100 pt**	100%	(15/15)
builder.exe	0%	(0/1)	0%	(0/3)	0.0 pt	0%	(0/4)	**50.00%**	**(6/12)**	**+50.00 pt**	50.00%	(6/12)
captcha.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/3)	**100%**	**(9/9)**	**+100 pt**	100%	(9/9)
cli.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/4)	**100%**	**(12/12)**	**+100 pt**	100%	(12/12)
clickfix.exe	0%	(0/1)	**33.33%**	**(1/3)**	**+33.33 pt**	0%	(0/5)	**40.00%**	**(6/15)**	**+40.00 pt**	40.00%	(6/15)
download1.exe	0%	(0/1)	**66.67%**	**(2/3)**	**+66.67 pt**	0%	(0/4)	**83.33%**	**(10/12)**	**+83.33 pt**	83.33%	(10/12)
download2.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/4)	**100%**	**(12/12)**	**+100 pt**	100%	(12/12)
installer.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/12)	**100%**	**(36/36)**	**+100 pt**	100%	(36/36)
macro_excel.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	100%	(3/3)
macro_word.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	100%	(3/3)
mail.exe	0%	(0/1)	**33.33%**	**(1/3)**	**+33.33 pt**	0%	(0/4)	**55.56%**	**(10/18)**	**+55.56 pt**	55.56%	(10/18)
ransomnote.exe	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	0%	(0/1)	**100%**	**(3/3)**	**+100 pt**	100%	(3/3)
uac_dialog.exe	100%	(1/1)	100%	(3/3)	0.0 pt	100%	(2/2)	100%	(6/6)	0.0 pt	100%	(6/6)
合計	**7.69%**	**(1/13)**	**79.49%**	**(31/39)**	**+71.79 pt**	**4.00%**	**(2/50)**	**83.97%**	**(131/156)**	**+79.97 pt**	**83.97%**	**(131/156)**

付録 A.1 に示す Algorithm 1 を実装した vlm_agent.py を auxiliary module（`/opt/CAPEv2/modules/auxiliary/`）に追加した．また，解析時に同モジュールを有効化するため，`/opt/CAPEv2/conf/auxiliary.conf` に設定を追加した．提案手法の各コンポーネントは，サーバー VM 上の `CAPE/lib/cuckoo/vlm_agent/` に配置した．これにより，従来の human.py によるイベント応答を提案手法に置換した．加えて，クライアント VM 側には，受信した操作列を PC 操作自動化ライブラリである pyautogui [14] により自動実行する executor.py を配置した．

5. 評価

5.1 実験環境設定

本実験は，4 章で述べた CAPE v2 上の実装環境を用いて実施した．クライアント VM の OS には Windows 10 Pro を用いた．解析開始時には試行ごとに同一のクリーンスナップショットへ復元し，初期状態を統一した．操作生成部には，VLM として Qwen3-VL [15] を用いて操作列生成を実施した．なお，本実験では模擬検体を用いることから他 VM に悪影響を及ぼす可能性がないため，安全性検査部にて静的フィルタのみを適用する．LLM フィルタの有効性に関する議論は付録 A.3 に示す．

5.2 評価データセット

本研究では，提案手法を評価するため，13 件の模擬検体からなる評価データセットを構築した．本データセットは，前段研究 [7] で確認された全 14 種類のイベント種別を含むよう設計し，同研究で調査対象とした ANY.RUN 解析レポート上のイベントを基に作成した．一部の模擬検体は複数のイベント種別を含むため，13 件の模擬検体で全 14 種類のイベント種別を網羅している．各模擬検体では，実際の悪性処理は含めず，後続の解析段階へ進むために必要となる対話的イベント，画面遷移，および操作手順を再現した．さらに，各模擬検体は，正しいイベント応答が行われた場合にのみ後続画面または完了状態へ遷移するよう設計し，実マルウェアにおいてイベント応答後に顕現する状態遷移を模擬した．各検体のイベント内容，正解操作，および再現元となったレポート URL を付録 A.4 に示す．

5.3 評価方法

本研究では，[16] を参考に，評価指標として Completion Rate，Progress Rate，Grounding Rate を用いる．Completion Rate は一連の操作を完了した試行の割合，Progress Rate は必要操作の達成割合，Grounding Rate は操作生成部が生成した操作列自体の正解割合を表す．

提案手法は，13 件の模擬検体に対して各 3 回，計 39 試行を実施した．これは，生成モデルの推論結果が同一条件でも変動し得るため，複数回実行による評価が必要であるとの先行研究 [17] の指摘を踏まえたものである．なお，VLM における操作生成時のパラメータは，出力のばらつきを抑えつつ一定の多様性を確保するために temperature を 0.2，top_p を 0.9 に設定した．一方，ルールベースの human.py は各検体 1 回のみ試行した．正解ラベルとして，解析に必要な操作列と完了時の画面状態を主著者が人手で作成した．同一の完了状態に至る妥当な経路が複数ある場合は，経路ごとに正解操作列を定義し，各試行に対応する正解操作列を用いて評価した．なお，いずれの経路でも進捗が得られない場合は，最短の正解操作列を基準として評価した．また，解析進行に影響しない余分な操作や待機は失敗とせず，誤った画面遷移，対象外へのクリック，誤入力，必要操作の欠落を失敗とした．解析完了後の操作は対象外とし，成否は主著者が人手で判定した．この方針により，提案手法では全 39 試行に対応する計 156 操作を評価対象とした．

また，提案手法では，複数ステップにわたるイベント応答で過去の実行状況を踏まえて後続操作を生成するため，操作内容，画面情報，および操作検証結果を履歴情報として後続のプロンプトに含める．これらの履歴情報がイベント応答性能に与える寄与を検証するため，アブレーション

© 2026 Information Processing Society of Japan

評価を実施した．アブレーション評価では，すべての履歴情報を含む提案手法を基準とし，操作内容，画面情報，操作検証結果をそれぞれ個別に除外した 3 条件と，すべてを除外した条件を設定した．各条件について，提案手法と同様に 13 件の模擬検体を各 3 回，計 39 試行実施した．

5.4 性能評価

表 3 に，全試行における性能評価結果を示す．本評価では，CAPE v2 の human.py との性能を比較した．human.py は，13 検体中 1 検体のみを完了し，Completion Rate は 7.69%，Progress Rate は 4.00%であった．達成した操作はダイアログ上の単純なボタンクリックに限られ，キー入力を伴う操作等には対応できなかった．一方，提案手法は 13 検体，39 試行中 31 試行で解析に必要な一連の操作を完了し，Completion Rate は 79.49%であった．また，正解操作として定義した 156 操作のうち，131 操作を正しく生成・実行し，Progress Rate および Grounding Rate はいずれも 83.97%であった．この結果から，提案手法は従来手法を大きく上回り，多様なイベントへ応答可能であることが確認された．なお，Grounding Rate と Progress Rate が同値であったことから，失敗事例は操作生成部が生成した操作列自体の誤りに起因したといえる．

図 2 に，提案手法による成功試行の例を示す．図 2（a）は download1.exe における成功例である．本検体では，初期画面に表示されたパスワードを読み取り，その情報を保持したうえで，画面遷移後の入力画面で正しく入力する必要がある．成功試行では，初期画面に表示されたパスワードが VLM の出力する画面情報に含まれ，履歴として保持された．その後，パスワード入力画面に遷移した際に，画面上にパスワードが明示されていなかったにもかかわらず，操作生成部は履歴中のパスワードを基にキー入力操作を生成し，ファイルのダウンロード操作を完了した．

図 2（b）は clickfix.exe における成功例である．本検体では，ClickFix 型画面を模擬しており，画面上に表示された手順説明を読み取り，複数の操作へ変換する必要がある．成功試行では，操作生成部が画面上の手順説明を読み取り，チェックボックスのクリック，Win+R，Ctrl+V，Enter の入力，Verify ボタンクリックを含む複数の操作列を手順通り実行し，解析に必要な操作を完了した．さらなる成功事例の定性評価は付録 A.5 に示す．

5.5 履歴情報に関するアブレーション評価

表 4 に，後続の操作生成で参照する履歴情報のアブレーション評価結果を示す．履歴をすべて除外した条件では，Completion Rate が 61.53%，Progress Rate が 68.59%となり，最も低い性能を示した．この結果から，履歴情報を後続の操作生成に反映することが，複数ステップにわたるイベント応答に有効であることが確認できた．各要素を個別に除外した場合，操作検証結果を除外した条件では Completion Rate が 74.35%，Progress Rate が 80.77%であった．これに対し，操作内容を除外した条件では 69.23%，73.72%，画面情報を除外した条件では 69.23%，75.64%となった．したがって，本実験では，操作検証結果よりも，画面情報および操作内容の欠落が性能低下に大きく影響することが確認された．画面情報を除外した条件では，download1.exe の 3 試行で，初期画面に表示されたパスワードが後続の

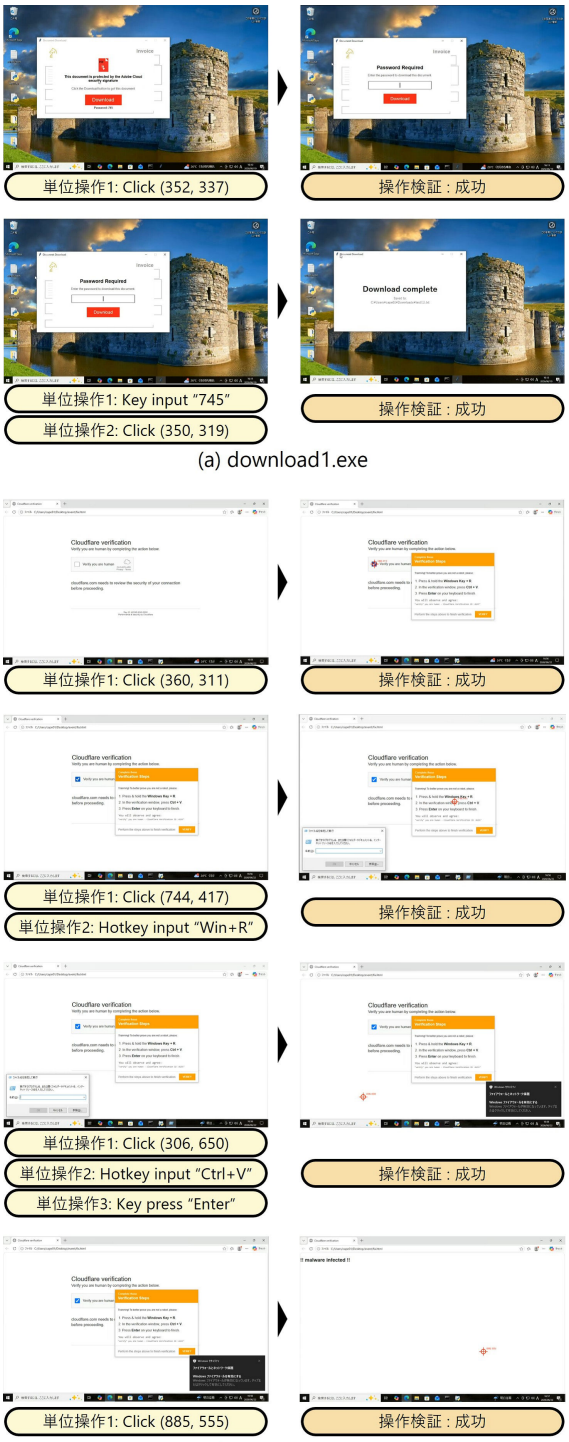


図 2 提案手法がイベント応答に成功した事例

© 2026 Information Processing Society of Japan

表 4　履歴情報の構成要素に関するアブレーション評価

評価条件	プロンプトに含まれる履歴情報			Completion Rate			Progress Rate			Grounding Rate		
	操作内容	画面情報	操作検証結果	割合	件数	差分	割合	件数	差分	割合	件数	差分
提案手法	✓	✓	✓	**79.49%**	**(31/39)**	—	**83.97%**	**(131/156)**	—	**83.97%**	**(131/156)**	—
w/o 操作内容	—	✓	✓	69.23%	(27/39)	**-10.26 pt**	73.72%	(115/156)	**-10.25 pt**	73.72%	(115/156)	**-10.25 pt**
w/o 画面情報	✓	—	✓	69.23%	(27/39)	**-10.26 pt**	75.64%	(118/156)	**-8.33 pt**	75.64%	(118/156)	**-8.33 pt**
w/o 操作検証結果	✓	✓	—	74.35%	(29/39)	**-5.14 pt**	80.77%	(126/156)	**-3.20 pt**	80.77%	(126/156)	**-3.20 pt**
w/o 全履歴情報	—	—	—	61.53%	(24/39)	**-17.96 pt**	68.59%	(107/156)	**-15.38 pt**	68.59%	(107/156)	**-15.38 pt**

入力画面へ引き継がれず失敗した．これは，画面情報の履歴が，後続画面では観測できない一時的な情報を保持する役割を持つことを示している．操作内容を除外した条件では，cli.exe や download1.exe で同じキー入力操作を繰り返す挙動が確認された．これは，直前の操作内容を参照できないことで実行済みの操作を再生成しやすくなるため，操作内容の履歴情報が，進行状況の把握と重複操作の抑制に寄与することを示している．操作検証結果を除外した場合も性能は低下したが，画面情報や操作内容を除外した場合と比べて影響は小さかった．これは，本実験で用いた模擬検体では，画面遷移が比較的単純であり，操作後の画面情報と実行済み操作から進行状況を推定しやすいためと考えられる．一方，実検体では，処理待ちや非同期な画面更新，複数ウィンドウの遷移，想定外のダイアログなどにより，操作結果が直ちに明確な画面変化として現れない場合が想定される．そのため，操作検証結果の有効性については，実検体を用いて今後評価する必要がある．

以上より，履歴情報を後続の操作生成に反映することは，複数画面にまたがるイベント応答を可能にし，その進行を安定化する上で有効である．特に，画面情報は後続画面では観測できない一時的な情報の保持に，操作内容は進行状況の把握と重複操作の抑制に寄与することが確認された．

6. 議論

6.1 従来手法の限界に対する提案手法の有効性

human.py のようなルールベース手法は，事前定義されたルールに基づくクリック操作を中心としており，ダイアログへのクリックには対応できる．一方，文字列入力，ファイル操作，コマンドライン操作などを要求する多様なイベントへの対応は困難である．

これに対し，提案手法は，VLM による画面理解とアクション空間に基づく操作列の生成により，画面文脈に応じた操作種別，操作対象および変数の決定が可能であることを確認した．例えば，download1.exe の成功試行では，画面遷移前に表示されたパスワードを基に後続画面での入力値として利用できた．この結果は，前段のスクリーンショットから取得された一時的な画面情報を履歴として保持し，後続の操作生成時に VLM から参照されることで，複数画面にまたがる操作を継続できる可能性を示している．また，clickfix.exe の成功試行では，画面上に自然言語で示された手順を解釈し，複数の操作からなる操作列へ変換できた．ClickFix 型の攻撃では，ユーザに対してショートカットキーを利用した一連の操作を要求する場合があり，本結果は，提案手法が画面上の自然言語による手順説明を解釈し，解析に必要な操作列を実行できる可能性を示している．

以上より，成功試行から確認された提案手法の有効性は，第一に，クリックに限定されない多様な操作を実行できる点，第二に，画面情報等の履歴情報を利用して複数画面にまたがる操作を継続できる点，第三に，画面上の表示内容や手順説明に基づいて操作内容を柔軟に決定できる点にある．これらにより，提案手法は，従来手法では対応困難な対話的イベントにも応答し，動的解析自動化のカバレッジを拡大できる可能性が示された．なお，当該有効性はすべての試行で確認されたものではないことに留意されたい．

6.2 失敗事例に基づく課題分析

本節では，失敗事例に基づいて課題を整理する．

後続操作に必要な情報や進行状況の抽出・保持能力の不足．

download1.exe の失敗試行では，VLM が初期画面のパスワードを認識できず，履歴に保持されなかった．そのため，後続の入力画面で必要な文字列を現在画面と履歴のいずれからも取得できず，“password” のキー入力操作を生成した．また，mail.exe の失敗試行では，添付ファイルの保存までは完了したものの，保存後にファイルマネージャを開いて実行する必要があると判断できなかった．その結果，操作対象がないとして待機を繰り返したほか，別の試行ではメール画面に戻り，添付ファイルを再ダウンロードした．これらの結果は，画面遷移を伴う操作を継続するには，後続操作に必要な情報を正確に抽出するとともに，操作の進行状況を継続的に把握し，操作生成へ反映する必要があることを示している．したがって，パスワードなど，後続操作で参照される可能性が高い情報は初回観測時に明示的に抽出し，ファイル保存など後続操作の前提となる操作は完了状況を履歴に保持したうえで，進行状況に基づいて操作を生成するようプロンプトを設計する必要がある．

クリック領域の座標推論の誤り．

clickfix.exe の失敗試行では，VLM は認証突破にチェックボックスのクリックが必要と認識していたが，本体ではなく隣接するラベル領域をクリックしたため，状態が変化せず後続操作へ進めなかった．この結果は，VLM による画面理解と，推論されたクリック座標が必ずしも整合しな

© 2026 Information Processing Society of Japan

いことを示している．対策として，スクリーンショットから座標を直接推論するのではなく，関連領域を抽出し，その領域を拡大した局所画像から座標を推論する二段階設計が考えられる．RegionFocus [18] では，関連領域の拡大により画面の視覚的複雑性が低減され，座標の推論精度向上に寄与することが示唆されている．この設計を導入することで，座標の誤推論を抑制できる可能性がある．

手順説明や設定項目を考慮した操作計画能力の不足．

clickfix.exe の失敗試行では，VLM は画面上の手順説明から Win+R や Ctrl+V のホットキー入力，Enter のキー押下が必要と認識していたにもかかわらず，これらを実行せず Verify ボタンをクリックした．また，ビルダーを模擬した builder.exe の 3 試行では，設定項目である Email Settings を確認せず直ちに Build ボタンをクリックした．実際には，前者では指示されたコマンドの実行後に Verify をクリックし，後者では情報送信先を設定してから Build を実行する必要がある．これらの結果は，提案手法が手順説明や設定項目を認識しても，解析の進行に直接関係する操作を優先し，その前提となる操作を欠落することを示している．対策として，画面から直ちに次の操作を出力するのではなく，まず手順説明や設定項目を含む画面全体の情報を，解析に必要なサブタスク列へ整理し，その順序に沿って原始的な操作列へ変換する構成が考えられる．VideoGUI [19] でも，このような段階的な操作生成の有効性が示唆されており，必要な手順や設定確認の欠落を抑制できる可能性がある．

7. 制約

本評価は，主著者が作成した 13 件の模擬検体を対象としており，実マルウェアが示す全イベントへの有効性を保証するものではない．一方，前段研究で整理した全 14 種のイベント種別を含む評価データセットを構築し，網羅性を確保した．さらに，同研究で調査した ANY.RUN 解析レポート上のイベントおよび操作要求を再現した模擬検体を用いることで，評価の妥当性を高めた．

8. おわりに

本稿では，マルウェア動的解析中に発生する対話的イベントへの自動応答のための VLM ベースエージェントを提案した．13 件の模擬検体を用いて CAPE v2 上で評価した結果，提案手法は 39 試行中 31 試行（79.49%）で一連の操作を完了し，従来手法では対応困難なイベントに応答できた．また，一部の試行で，画面遷移前に取得した画面情報や前ステップの操作内容を履歴情報として保持し，後続画面での操作生成に反映することで，複数画面にまたがるイベント応答を可能にした．また，失敗事例分析から，提案手法の課題を 3 つ抽出し，研究可能性を導出した．今後は，実マルウェアを用いた評価を通して有効性を検証する．

謝辞 本研究の遂行にあたり，日立グループ内のサイバー攻撃解析に関わる佐藤隆行氏や専門家各位に有益な助言とご協力を頂きました．深く感謝致します．

参考文献

[1] Wong, M. Y. et al.: An Inside Look into the Practice of Malware Analysis, *CCS '21*, pp. 3053–3069 (2021).
[2] GitHub: kevoreilly/CAPEv2, `https://github.com/kevoreilly/CAPEv2`. Accessed: 2026-07-21.
[3] Lengyel, T. K. et al.: Scalability, Fidelity and Stealth in the DRAKVUF Dynamic Malware Analysis System, *ACSAC '14*, pp. 386–395 (2014).
[4] Gruber, J. et al.: Fighting Evasive Malware: How to Pass the Reverse Turing Test by Utilizing a VMI-Based Human Interaction Simulator, *Datenschutz und Datensicherheit*, Vol. 46, pp. 284–290 (2022).
[5] Joe Security LLC: Joe Sandbox Cloud Basic, `https://www.joesandbox.com/`. Accessed: 2026-07-21.
[6] Joe Security LLC: LEVEL UP: Detecting Phishing with GenAI, `https://joesecurity.org/blog/6811663389969520216`. Accessed: 2026-07-21.
[7] 塙ほか: マルウェア動的解析で要求される対話的イベントの抽出と LLM による自動応答可能性の検討, *CSS '25*, pp. 1691–1698 (2025).
[8] ANY.RUN: ANY.RUN, `https://any.run/`. Accessed: 2026-07-21.
[9] Niu, R. et al.: ScreenAgent: A Vision Language Model-driven Computer Control Agent, *IJCAI '24*, pp. 6433–6441 (2024).
[10] Lin, K. et al.: ShowUI: One Vision-Language-Action Model for GUI Visual Agent, *CVPR '25*, pp. 19498–19508 (2025).
[11] Yang, J. et al.: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, *NeurIPS '24*, pp. 50528–50656 (2024).
[12] Hua, W. et al.: TrustAgent: Towards Safe and Trustworthy LLM-based Agents, *EMNLP '24*, pp. 10000–10016 (2024).
[13] Gao, D. et al.: ASSISTGUI: Task-Oriented Desktop Graphical User Interface Automation, *CVPR '24*, pp. 13289–13298 (2024).
[14] GitHub: pyautogui, `https://github.com/asweigart/pyautogui`. Accessed: 2026-08-15.
[15] Ollama: Qwen3-VL, `https://ollama.com/library/qwen3-vl`. Accessed: 2026-07-01.
[16] Ma, C. et al.: AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents, *NeurIPS '24*, pp. 74325–74362 (2024).
[17] Bjarnason, B. H. et al.: On Randomness in Agentic Evals, *ICLR Workshop '26* (2026).
[18] Luo, T. et al.: Visual Test-time Scaling for GUI Agent Grounding, *ICCV '25*, pp. 19989–19998 (2025).
[19] Lin, K. Q. et al.: VideoGUI: A Benchmark for GUI Automation from Instructional Videos, *NeurIPS '24*, pp. 69329–69360 (2024).
[20] Bailey, M. et al.: The Menlo Report, Technical report, U.S. Department of Homeland Security (2012).
[21] Ollama: gpt-oss:20b, `https://ollama.com/library/gpt-oss:20b`. Accessed: 2026-07-15.

© 2026 Information Processing Society of Japan

倫理的配慮

本研究では，マルウェア動的解析における対話的イベント応答の自動化を目的としており，攻撃機能の開発やマルウェアの改変・拡散を目的としない．

実験にあたっては，メンロレポート [20] を参照した．実験では，実マルウェアではなく，前段の研究 [7] で確認された対話的イベントを再現した模擬検体を用いた．これにより，実マルウェアで想定される外部環境への通信，感染拡大などのリスクを避けつつ，提案手法が対話的イベントへ応答できるかを妥当に評価した．また，本エージェントは解析を進行させるための操作を自動生成・実行するため，実マルウェアを対象とする運用では，外部通信，認証情報入力，ネットワーク設定変更など，解析環境外へ影響し得る危険性を有する操作を生成する可能性がある．このため，実マルウェアの動的解析への適用を見越して，本研究では安全性検査部での LLM フィルタによる意味的な操作列判定を導入した．付録 A.3 にて定性評価を実施し，解析環境外へ影響し得る危険性のある操作を抑制できる見込みを示した．

付　　録

A.1　提案手法のオーケストレーション手順

Algorithm 1 に，提案手法におけるオーケストレーション手順の擬似コードを示す．本アルゴリズムでは，画面観測，操作生成，安全性検査，操作実行，操作検証，および履歴更新からなる一連の処理を繰り返す．この反復処理は，最大ステップ数への到達，安全性検査または操作結果検証の連続失敗回数が所定の上限に達した場合，あるいは CAPE v2 側で解析の終了が判定された場合に終了する．

Algorithm 1 オーケストレーション手順

Require: 最大ステップ数 $T_{\max}$，安全性検査の連続失敗上限 C_{safety}，操作検証の連続失敗上限 C_{verify}
履歴管理部 SM を初期化
失敗回数を初期化：$c_{\text{safety}} \leftarrow 0$，$c_{\text{verify}} \leftarrow 0$
ステップ数を初期化：$t \leftarrow 0$

function 停止条件 $(SM, c_{\text{safety}}, c_{\text{verify}}, t)$
  return $(t \geq T_{\max}) \vee (c_{\text{safety}} \geq C_{\text{safety}}) \vee (c_{\text{verify}} \geq C_{\text{verify}}) \vee$ CAPE v2 終了判定
end function

while 停止条件 $(SM, c_{\text{safety}}, c_{\text{verify}}, t)$ が偽 **do**
  $(screenshot_t^{\text{before}}, cursor_t^{\text{before}}) \leftarrow$ 画面観測
  $SM \leftarrow$ 観測結果更新 $(SM, screenshot_t^{\text{before}}, cursor_t^{\text{before}})$
  $prompt_t \leftarrow$ プロンプト生成 $(template, screenshot_t^{\text{before}}, SM)$
  $(action_list_t, screen_info_t) \leftarrow$ 操作生成 $(prompt_t)$
  $SM \leftarrow$ 操作列更新 $(SM, action_list_t, screen_info_t)$
  $safety_result_t \leftarrow$ 安全性検査 $(action_list_t, screen_info_t)$
  $SM \leftarrow$ 安全性検査結果更新 $(SM, safety_result_t)$
  if $safety_result_t = \text{fail}$ **then**
    $c_{\text{safety}} \leftarrow c_{\text{safety}} + 1$
    $t \leftarrow t + 1$
    次のステップへ
  else
    $c_{\text{safety}} \leftarrow 0$
  end if
  for all $subaction \in$ 操作制御 $(action_list_t)$ **do**
    操作実行 $(subaction)$
    $SM \leftarrow$ 実行結果更新 $(SM, subaction)$
  end for
  $(screenshot_t^{\text{after}}, cursor_t^{\text{after}}) \leftarrow$ 画面観測
  $SM \leftarrow$ 観測結果更新 $(SM, screenshot_t^{\text{after}}, cursor_t^{\text{after}})$
  $verify_result_t \leftarrow$ 操作検証 $(screenshot_t^{\text{before}}, cursor_t^{\text{before}}, screenshot_t^{\text{after}}, cursor_t^{\text{after}})$
  $SM \leftarrow$ 操作検証結果更新 $(SM, verify_result_t)$
  if $verify_result_t = \text{fail}$ **then**
    $c_{\text{verify}} \leftarrow c_{\text{verify}} + 1$
  else
    $c_{\text{verify}} \leftarrow 0$
  end if
  $t \leftarrow t + 1$
end while

A.2　プロンプトの詳細

図 A·1 に，プロンプト生成部で事前定義したプロンプトのテンプレートを示す．本プロンプトでは，VLM に対して，マルウェア動的解析中に表示される画面を観測し，解析を進行させるために必要な操作列を生成するよう指示した．操作対象は現在の画面上に限定し，将来の画面遷移や画面外の情報を推測せず，現在実行可能な操作を優先する等の操作生成ルールを定めた．また，本研究で定義したアクション空間に含まれる 8 つの操作内容を定めた．出力は，操作種別，変数，画面内容の説明，操作対象の説明，および操作理由を含む JSON 配列に限定した．

A.3　安全性検査部における LLM フィルタの定性評価

本研究の定量評価では，実マルウェアではなく模擬検体を用いたため，安全性検査部には静的フィルタのみを適用した．しかし，静的フィルタは，操作列が解析環境外へ影響を及ぼす可能性があるかという意味的な危険性までは判定できない．そこで，本節では，実マルウェアでの評価に向けて，安全性検査部における LLM フィルタの定性的評価を行い，LLM フィルタの有効性を分析する．LLM フィルタでは，VLM が生成した操作列に加えて，画面情報，操作対象，操作理由を入力として，当該操作がクライアント環境外のシステム，サービス，または他環境に影響を及ぼす可能性があるかを判定する．本研究では，外部サービスや外部アカウントへの接続，外部サービスに対する認証情報入力，外部ネットワークへのデータ送信，同期・アップロード・共有の開始，ネットワーク設定変更，およびサ

© 2026 Information Processing Society of Japan

表 A·1　プロンプトテンプレート

Prompt

```
You are an assistant that observes GUI screens shown
during dynamic malware analysis and determines the
sequence of user actions required to advance the analysis.
The attached image contains a GUI screen captured during
dynamic malware analysis.  Examine the image and determine
the user action or sequence of actions required to proceed
with the analysis.
A “required user action” means the most appropriate
interaction with the GUI elements currently visible in
the image that would help the analysis continue.  Do not
consider future screen transitions or anything outside
the current image.  Prioritize concrete GUI actions
that are necessary to advance the malware analysis,
such as clicking a visible button, focusing an input
field, entering required text, pressing a required key, or
scrolling a required area.  Use wait only as a fallback
action when there is no visible, useful, and safe GUI
interaction that should be performed on the current
screen.  Do not choose wait if a specific visible GUI
action is required to advance the analysis.  If no useful
or safe GUI interaction is required on the current screen,
output a wait action.  Your output must strictly follow
the JSON array format below.  Do not output any text other
than JSON. Do not include explanations, preambles, code
fences, or Markdown.

Constraints:
- Use only the defined action_type values.
(omitted:  additional constraints on bbox validation,
target-label inclusion, JSON validity, minimal action
sequences, and prohibited extra fields)

Format:  [ { "action":  { "action_type":  "Click |
double-click | right-click | key-input | key-press |
hotkey-input | scroll | wait", "bbox":  [x_min, y_min,
x_max, y_max], "key":  "<key or text>", "direction":
"up | down" }, "meta":  { "screen_text":  "<screen
description>", "target_text":  "<target description>",
"reason":  "<reason for the action>" } } ]

Example:
[omitted:  additional few-shot examples for button click,
key press, scroll, hotkey input, and wait]
```

ンドボックス隔離を弱める操作を危険な操作として拒否ポリシーを定義した．一方で，UAC 承認，Enable Content，ローカルダイアログへの応答などは，外部影響を示す文脈がない限り，それ自体では危険とは判定しない方針とし，許可ポリシーを定義した．

図 A·1 に，gpt-oss:20b[21] を用いた LLM フィルタが静的フィルタでは検出できない意味的な危険性を検出した例を示す．図 A·1（a）は captcha.exe における CAPTCHA 入力画面であり，VLM は CAPTCHA 入力欄へのクリック，CAPTCHA 文字列の入力，Verify ボタンのクリックから成る操作列を生成した．この操作列に対して，静的フィルタは操作列形式，操作種別，および座標範囲が妥当であるため許可と判定した．一方で，LLM フィルタは，VLM の出力に含まれる，“Verify ボタンをクリックすると，CAPTCHA の回答が送信され，解析が進行する” 等の文言を参照し，Verify ボタンのクリックが外部サービスまたはプラットフォーム側の CAPTCHA 検証を開始する可能性があるとして拒否と判定した．図 A·1（b）は clickfix.exe における Cloudflare 認証の画面であり，VLM は，画面上の「Verify you are human」チェックボックスをクリックする操作列を生成した．この操作列についても，静的フィルタは，操作列の形式および座標範囲が妥当であるため許可と判定した．一方で，LLM フィルタは，VLM の出力に含まれる，“解析を進めるには，セキュリティ認証を通過するため 「Verify you are human」 チェックボックスをクリックする必要がある” 等の文言を参照し，当該操作が外部サービスへの接続開始に関与する可能性があるとして拒否と判定した．これらの事例は，静的フィルタと LLM フィルタの役割の違いを示している．静的フィルタは，操作種別や座標範囲の妥当性を検査できる一方で，同じクリック操作が解析環境内での画面遷移を目的とするのか，外部サービスとの通信や検証を伴う可能性があるのかを区別できない．これに対し，LLM フィルタは，VLM が出力した画面情報，操作内容，判断根拠を考慮することで，操作列の形式だけでは判断できない潜在的な外部影響を検出し，意味的な危険性を判断できる可能性がある．特に，CAPTCHA 検証や Cloudflare 認証のように，画面上の UI 操作が外部サービス側の検証処理と結び付いている可能性がある場合，LLM フィルタは保守的に拒否と判定することで，クライアント環境外への影響を抑制する役割を果たす可能性を示している．

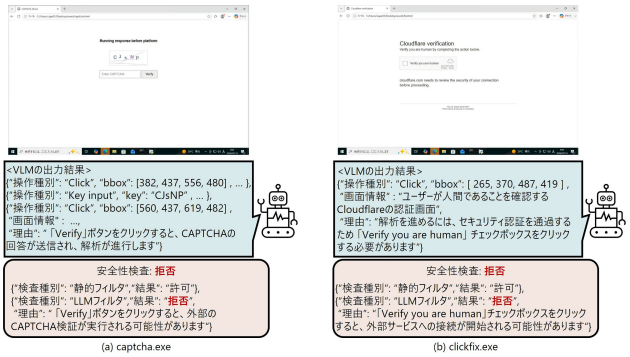


図 A·1　安全性検査部における LLM フィルタでの判定例

一方で，本実験では模擬検体を用いており，実際の外部サービスへの通信を伴うものとして評価したわけではない．したがって，本節で示した LLM フィルタの判定は，実際に外部通信が発生したことを確認したものではなく，画面文脈に基づいて潜在的な外部影響を検出した例として位置付ける．これは，LLM フィルタの有効性と同時に課題も示しており，LLM フィルタは静的フィルタでは検出でき

© 2026 Information Processing Society of Japan

表 A·2　評価データセットの詳細

評価検体	イベント種別	模擬イベント	正解操作の概要	再現元のイベントを含む ANY.RUN レポートの URL 末尾
analysis.exe（イベント数：4, 操作数：5）	解析アプリ, ファイルマネージャ	解析アプリにおけるファイル選択および処理選択	解析アプリ上でファイルを選択し，実行する処理に対応するボタンをクリックする	10b4d9f4-8cfb-4562-abad-2b0a74f9167e
builder.exe（イベント数：4, 操作数：4）	マルウェア関連アプリ，ダイアログ	情報窃取型マルウェアのビルダー	設定項目を変更し，「Build」ボタンをクリックする	0b675ab1-31f1-4a35-9089-a4fcb3884662
captcha.exe（イベント数：1, 操作数：3）	ブラウザ	画面に表示された文字列の入力をユーザに要求する認証	入力欄をクリックし，表示された文字列を入力した後，「Verify」ボタンをクリックする	6e02b4ec-e6c2-4e28-84ac-2e4d1cf80278
cli.exe（イベント数：2, 操作数：4）	コマンドライン	メニュー選択および Enter キー入力を要求するコマンドライン	コンソールに表示されたメニュー項目に対応するキーを入力し，Enter キーを押す	7123636c-f3e7-4141-befc-32eabb647836
clickfix.exe（イベント数：4, 操作数：5）	ブラウザ, OS 標準機能	複数の手順説明を有する ClickFix 型の認証	チェックボックスをクリックし，Win+R，Ctrl+V，Enter の順にキー入力する	92b02978-aabf-4ead-87c3-2d64eee2bafa
download1.exe（イベント数：3, 操作数：4）	アプリケーション	パスワードで保護されたファイルのダウンロード	「Download」ボタンをクリックし，画面に表示されたパスワードを入力した後，再度「Download」ボタンをクリックする	022e148d-6da6-4344-9e4a-1f8d9f06f1b5
download2.exe（イベント数：4, 操作数：4）	ブラウザ, ブラウザメニュー, ダイアログ	ブラウザからの実行ファイルのダウンロード，保存操作，ファイル実行，および実行確認	ダウンロードボタンと保存ボタンを順にクリックする．その後，ダウンロードしたファイルを実行し，実行確認ダイアログの「Yes」ボタンをクリックする	01a74e93-a7c1-4aca-97c7-28add55a1dcc
installer.exe（イベント数：12, 操作数：12）	デスクトップ，ファイルマネージャ, インストーラー	ZIP ファイルの展開後に起動し，複数回のボタンクリックを要求するインストーラー	実行ファイルを開き，「Next」,「Install」,「Finish」ボタンを順にクリックする	082ee721-f557-47df-a727-12f2374f43b7
macro_word.exe（イベント数：1, 操作数：1）	マクロ付きファイル	マクロ機能を有する Word ファイル	Word 上に表示される「コンテンツの有効化」ボタンをクリックする	5ca6499e-b634-4bef-a63c-4b65376d3032
macro_excel.exe（イベント数：1, 操作数：1）	マクロ付きファイル	マクロ機能を有する Excel ファイル	Excel 上に表示される「コンテンツの有効化」ボタンをクリックする	0c08c3f6-b8f0-4cf8-8e64-ef1e57f38ab1
mail.exe（イベント数：4/7, 操作数：4/7）	メール	ファイルが添付されたメール	Outlook 上でファイルが添付されたメールからファイルを開く，または保存後に開く	2e9bc49c-7d9b-48e2-a28a-b04b772509bf
ransomnote.exe（イベント数：1, 操作数：1）	デスクトップ	デスクトップ背景上で，ユーザに README ファイルを開くよう要求するランサムノート	デスクトップ背景で指定された対象ファイルをダブルクリックする	d520bea2-4ffe-4601-aecf-1955c61678ca
uac_dialog.exe（イベント数：2, 操作数：2）	ダイアログ	UAC 確認画面に続いて表示されるマルウェア実行確認	「Yes」ボタンをクリックする	ca9af122-8d6e-4a03-85a8-21cf103bb882

ない意味的な危険性を判断できる一方で，画面文脈を保守的に解釈することで，解析に必要な操作を拒否する可能性もある．したがって，LLM フィルタを実運用に導入する際には，危険操作を許可してしまう誤許可だけでなく，解析に必要な操作を拒否してしまう誤拒否も評価する必要がある．特に，CAPTCHA など，外部サービスや通信を想起させる語を含む画面では，LLM フィルタが安全側に倒れやすい可能性が本評価により示唆される．今後は，実検体を用いた評価データセットを準備し，LLM フィルタの誤許可・誤拒否を定量的に評価する必要がある．

A.4　評価データセットの詳細

表 A·2 に，本研究で用いた模擬検体の詳細を示す．各検体は，前段の研究 [7] で確認された対話的イベントを基に，実際のマルウェア動的解析中に表示された画面および解析に必要な操作を再現した模擬検体である．本表では，検体ごとに，イベント種別，模擬イベント，正解操作の概要，および再現元となるイベントを含む ANY.RUN レポートの URL 末尾を示す．

© 2026 Information Processing Society of Japan

A.5 提案手法による成功事例の定性評価

図 A·2 に，提案手法による成功事例を示す．提案手法がどのような画面情報を解釈し，どのような操作列を生成・実行したかに着目し，成功事例を定性評価する．

図 A·2（a）は，uac_dialog.exe における成功例である．本検体では，UAC 確認画面に対して許可する操作を行う必要がある．提案手法は，確認ダイアログ上に記載された文言およびボタンを認識し，Yes ボタンのクリック操作を実行できた．この結果は，提案手法が従来手法でも対象となる単純なダイアログ応答に対応できることを示している．

図 A·2（b）は，captcha.exe における成功例である．本検体では，ブラウザ上に表示された文字列を読み取り，入力欄をクリックした後，読み取った文字列を入力し，Verify ボタンをクリックする必要がある．提案手法は，画面上に表示された文字列を正確に認識し，入力欄に対してクリックを実行した後，認識した文字列をキー入力できた．この結果は，提案手法が単純なボタンクリックだけでなく，画面内に表示された一時的な文字情報を読み取り，以降の操作における入力値として反映できることを示している．また，キー入力操作の前に，入力欄をアクティブにするためのクリック操作が実行されており，クリック操作とキー入力操作を組み合わせた操作列の実行が可能であることが確認できた．この結果は，入力フォームを伴う対話的イベントに対する応答が可能であることを示している．

図 A·2（c）は，macro_excel.exe における成功例である．本検体では，マクロ付き Excel ファイルにて表示された英語の誘導文を解釈し，セキュリティ警告バーのボタンをクリックする必要がある．誘導文では “Enable editing” および “Enable content” のクリックが指示されたが，実際の画面上には編集の有効化に対応するボタンは表示されておらず，「コンテンツの有効化」ボタンのみが表示される．提案手法は，現在画面における警告バーを認識し，表示されていない “Enable editing” に対応する操作を不要と判断したうえで，英語の “Enable content” という指示を日本語 UI 上の「コンテンツの有効化」ボタンに対応付け，クリックを実行できた．この結果は，提案手法が文書内の誘導文を単純に逐語的に実行するのではなく，画面に存在する要素と照合しながら必要な操作を実行できることを示している．さらに，単純なボタンラベルの一致に基づいて操作対象を選択するのではなく，指示文と UI ラベルの言語差を吸収した上で操作対象を特定できることを示している．

図 A·2（d）は，mail.exe における成功例である．本検体では，メール本文に添付されたファイルを認識し，当該ファイルを開いた後，表示されたエクスプローラー画面上で解析対象となる実行ファイルを特定して実行する必要がある．提案手法は，メール本文および添付欄の表示内容から Enclosed_TT.zip を操作対象として特定し，ダブルクリック操作により添付ファイルを開くことができた．続いて，表示されたファイルマネージャ上で，アーカイブ内に含まれる dialog.exe を認識し，ダブルクリック操作により実行した．この結果は，提案手法がメールクライアント上の添付ファイルを認識できるだけでなく，メール画面からファイルマネージャ画面へ遷移した後も，画面状態に応じて操作対象を再特定し，メール添付型検体の解析を進行させる操作列を生成できることを示している．

図 A·2（e）は，cli.exe における成功例である．本検体では，CLI 画面上にロシア語で表示されたメニューを読み取り，適切な選択肢番号を入力し，処理を進める必要がある．提案手法は，まず CLI ウィンドウをクリックして入力対象をアクティブ化し，続いて選択肢番号「1」を入力し，最後に Enter キーを押下するという複数の操作を実行することができた．この結果は，提案手法がボタンやアイコンを対象とする典型的な操作だけでなく，CLI のようなテキストベースのインターフェースにも応答可能であることを示している．

図 A·2（f）は，ransomnote.exe における成功例である．本検体では，デスクトップ背景に表示された指示文を読み取り，該当するファイルをデスクトップ上から特定して開く必要がある．提案手法は，背景画面中の “Find i69h210-readme.txt and follow instructions” という指示を認識し，デスクトップ上に該当する txt ファイルに対して，ダブルクリック操作を実行し，正常にファイルを開くことができた．この結果は，提案手法がブラウザ上の入力欄のような明示的な要素だけでなく，デスクトップ背景に記載された指示文を解釈し，指示文に従ったファイル操作を実行できることを示している．

図 A·2（g）は，installer.exe における成功例である．本検体では，まずデスクトップ上から対象ファイルを特定し，ファイルマネージャ上で当該ファイルを選択した後，ファイルを実行し，インストーラーの指示に従ってインストールを完了する必要がある．提案手法は，デスクトップ上の対象ファイルを認識し，ダブルクリックによりフォルダを開き，続いてファイルマネージャ上で実行ファイルを右クリックし，表示されたコンテキストメニューから適切な項目を選択することで，インストーラーを起動できた．さらに，インストーラー起動後には，インストール処理進行のための応答だけでなく，インストール進行中の進捗バーを認識して待機操作を選択できた．そのため，余分な操作を行わずに待機し，インストール完了後には再び必要なボタン操作を実行した．この結果は，提案手法が単発のボタン操作にとどまらず，複数の画面種別をまたぐ長い操作列に対して，画面状態に応じて待機操作を適切に実施しながら処理を進行できることを示している．

© 2026 Information Processing Society of Japan

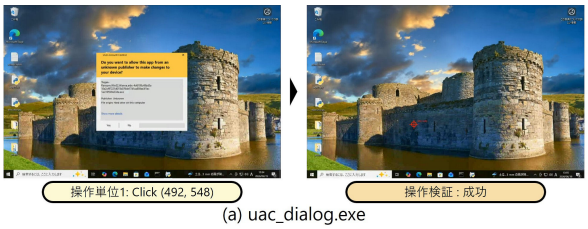


図 A·2 提案手法がイベント応答に成功した事例

© 2026 Information Processing Society of Japan